

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЁТ

по лабораторной работе № 1

Название: «Численное интегрирование»

Дисциплина: «Вычислительные алгоритмы»

Студент ИУ7-41Б _____ И.И. Иванов
(Группа) (Подпись, дата) (И.О. Фамилия)

Преподаватель _____ П.П. Петров
(Подпись, дата) (И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Теоретическая часть **4**

1.1 Постановка задачи 4

1.2 Квадратурные формулы 4

1.2.1 Формула средних прямоугольников 4

1.2.2 Формула трапеций 4

1.2.3 Формула Симпсона 5

1.3 Оценка погрешности 5

2 Практическая часть **7**

2.1 Реализация 7

2.2 Тестирование 7

2.3 Исследование погрешности 7

ЗАКЛЮЧЕНИЕ **11**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **12**

ПРИЛОЖЕНИЕ А **13**

ВВЕДЕНИЕ

Многие определённые интегралы, возникающие в инженерных расчётах, не выражаются через элементарные функции или выражаются слишком громоздко. Кроме того, подынтегральная функция часто задана не формулой, а таблицей значений, полученных измерениями или другой программой. В таких случаях интеграл вычисляют приближённо по значениям функции в отдельных точках [1].

Целью работы является сравнение составных квадратурных формул средних прямоугольников, трапеций и Симпсона по точности.

Для достижения поставленной цели необходимо решить следующие задачи:

1) описать квадратурные формулы и оценки их погрешности;

2) реализовать формулы и проверить реализацию на функциональных тестах;

3) вычислить погрешность формул для нескольких функций при разном числе отрезков разбиения;

4) сравнить полученную погрешность с теоретическими оценками.

1 Теоретическая часть

1.1 Постановка задачи

Требуется вычислить определённый интеграл, приведённый в формуле (1):

$$I = \int_a^b f(x)dx, \tag{1}$$

где $f(x)$ — функция, непрерывная на отрезке $[a, b]$ вместе с несколькими производными.

Отрезок $[a, b]$ делится на n равных частей точками $x_i = a + ih$, где $i = 0, 1, ..., n$, а $h = (b - a)/n$ — шаг разбиения. На каждой части функция заменяется более простой, интеграл от которой вычисляется точно, и результаты складываются [2]. Такие формулы называются составными.

1.2 Квадратурные формулы

1.2.1 Формула средних прямоугольников

На каждой части функция заменяется постоянной, равной её значению в середине части:

$$I \approx h \sum_{i=0}^{n-1} f\left(x_i + \frac{h}{2}\right). \tag{2}$$

1.2.2 Формула трапеций

На каждой части функция заменяется отрезком прямой, проходящей через значения функции на концах части:

$$I \approx h \left(\frac{f(x_0) + f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right). \tag{3}$$

1.2.3 Формула Симпсона

Части объединяются попарно, и на каждой паре функция заменяется параболой, проходящей через три точки. Число частей n при этом должно быть чётным:

$$I \approx \frac{h}{3}\left(f_0 + 4\sum_{i=1}^{n/2} f_{2i-1} + 2\sum_{i=1}^{n/2-1} f_{2i} + f_n \right), \tag{4}$$

где f_i — значение функции в точке x_i;

$n/2$ — число пар частей, на каждой из которых строится своя парабола.

Геометрический смысл трёх формул показан на рисунке 1. Площадь под кривой заменяется суммой площадей прямоугольников, трапеций или криволинейных трапеций под параболами.

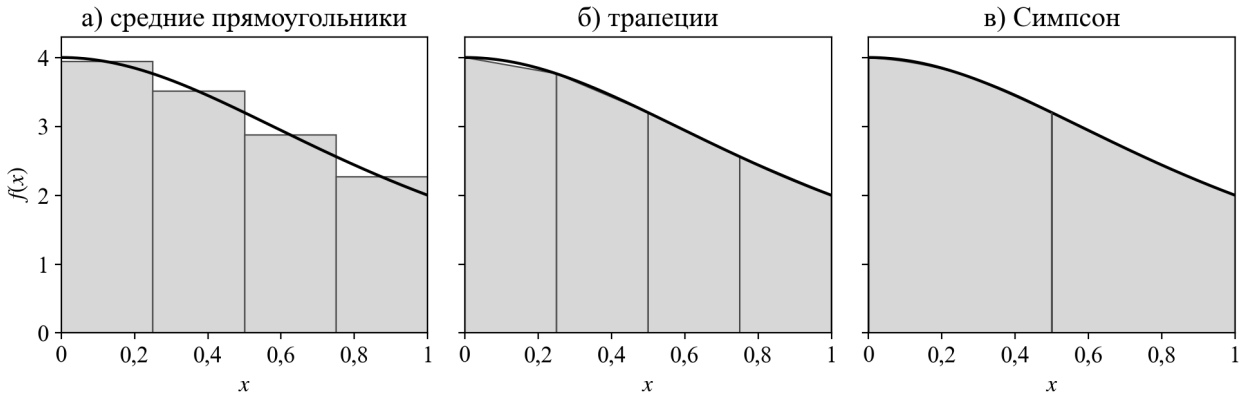


Рисунок 1 — Замена подынтегральной функции в квадратурных формулах при $n = 4$

1.3 Оценка погрешности

Погрешность составных формул убывает с уменьшением шага как степень h [1]. Показатель этой степени называется порядком точности формулы. Порядок и степень многочленов, которые формула интегрирует точно, приведены в таблице 1.

Таблица 1 — Порядок точности квадратурных формул

Формула	**Порядок точности**	**Степень точно интегрируемых многочленов**
Средних прямоугольников	2	1
Трапеций	2	1
Симпсона	4	3

Главный член погрешности формулы средних прямоугольников вдвое меньше по модулю, чем у формулы трапеций, и противоположен ему по знаку. Поэтому при одинаковом шаге формула средних прямоугольников обычно оказывается вдвое точнее.

Вывод

Формулы средних прямоугольников и трапеций имеют второй порядок точности, а формула Симпсона — четвёртый. Поэтому при уменьшении шага вдвое погрешность первых двух формул должна уменьшаться примерно в 4 раза, а формулы Симпсона — в 16 раз.

2 Практическая часть

2.1 Реализация

Формулы реализованы на языке Python [3]. Реализация формулы Симпсона приведена на листинге 1, полный текст программы — в приложении А.

```
1 def simpson(f: Function, a: float, b: float, n: int) -> float:
2     """Составная формула Симпсона на n отрезках, n должно быть чётным."""
3     if n % 2 != 0:
4         raise ValueError(f"для формулы Симпсона нужно чётное число отрезков, получено {n}")
5     h = (b - a) / n
6     odd = sum(f(a + i * h) for i in range(1, n, 2))
7     even = sum(f(a + i * h) for i in range(2, n, 2))
8     return h / 3 * (f(a) + f(b) + 4 * odd + 2 * even)
```

Листинг 1 — Реализация формулы Симпсона

2.2 Тестирование

Каждая формула интегрирует точно многочлены степени, указанной в таблице 1, поэтому на них результат должен совпасть с точным значением до ошибки округления. Тесты запускаются ключом `--test` и проверяют следующие случаи:

— формулы средних прямоугольников и трапеций на линейной функции;

— формулу Симпсона:

а) на кубической функции при наименьшем допустимом числе частей;

б) на квадратичной функции при четырёх частях;

в) на нечётном числе частей, при котором программа должна сообщить об ошибке.

Все тесты пройдены.

2.3 Исследование погрешности

Погрешность вычислялась для четырёх функций с известным значением интеграла:

$$\int_0^1 \frac{4}{1+x^2}dx = \pi, \quad \int_0^\pi \sin x dx = 2, \quad \int_0^1 e^x dx = e - 1, \quad \int_0^1 \sqrt{x}dx = \frac{2}{3}.$$

Число частей менялось от 2 до 1024 удвоением. Результаты приведены в таблице 2.

Таблица 2 — Абсолютная погрешность квадратурных формул

Функция	n	**Прямоугольники**	**Трапеции**	**Симпсон**
$4/(1+x^2)$	2	$2{,}076 \cdot 10^{-2}$	$4{,}159 \cdot 10^{-2}$	$8{,}259 \cdot 10^{-3}$
	4	$5{,}208 \cdot 10^{-3}$	$1{,}042 \cdot 10^{-2}$	$2{,}403 \cdot 10^{-5}$
	8	$1{,}302 \cdot 10^{-3}$	$2{,}604 \cdot 10^{-3}$	$1{,}511 \cdot 10^{-7}$
	16	$3{,}255 \cdot 10^{-4}$	$6{,}510 \cdot 10^{-4}$	$2{,}365 \cdot 10^{-9}$
	32	$8{,}138 \cdot 10^{-5}$	$1{,}628 \cdot 10^{-4}$	$3{,}696 \cdot 10^{-11}$
	64	$2{,}035 \cdot 10^{-5}$	$4{,}069 \cdot 10^{-5}$	$5{,}773 \cdot 10^{-13}$
	128	$5{,}086 \cdot 10^{-6}$	$1{,}017 \cdot 10^{-5}$	$8{,}882 \cdot 10^{-15}$
	256	$1{,}272 \cdot 10^{-6}$	$2{,}543 \cdot 10^{-6}$	0
	512	$3{,}179 \cdot 10^{-7}$	$6{,}358 \cdot 10^{-7}$	0
	1024	$7{,}947 \cdot 10^{-8}$	$1{,}589 \cdot 10^{-7}$	0
$\sin x$	2	$2{,}214 \cdot 10^{-1}$	$4{,}292 \cdot 10^{-1}$	$9{,}440 \cdot 10^{-2}$
	4	$5{,}234 \cdot 10^{-2}$	$1{,}039 \cdot 10^{-1}$	$4{,}560 \cdot 10^{-3}$
	8	$1{,}291 \cdot 10^{-2}$	$2{,}577 \cdot 10^{-2}$	$2{,}692 \cdot 10^{-4}$
	16	$3{,}216 \cdot 10^{-3}$	$6{,}430 \cdot 10^{-3}$	$1{,}659 \cdot 10^{-5}$
	32	$8{,}034 \cdot 10^{-4}$	$1{,}607 \cdot 10^{-3}$	$1{,}033 \cdot 10^{-6}$
	64	$2{,}008 \cdot 10^{-4}$	$4{,}016 \cdot 10^{-4}$	$6{,}453 \cdot 10^{-8}$
	128	$5{,}020 \cdot 10^{-5}$	$1{,}004 \cdot 10^{-4}$	$4{,}032 \cdot 10^{-9}$
	256	$1{,}255 \cdot 10^{-5}$	$2{,}510 \cdot 10^{-5}$	$2{,}520 \cdot 10^{-10}$
	512	$3{,}137 \cdot 10^{-6}$	$6{,}275 \cdot 10^{-6}$	$1{,}575 \cdot 10^{-11}$
	1024	$7{,}844 \cdot 10^{-7}$	$1{,}569 \cdot 10^{-6}$	$9{,}841 \cdot 10^{-13}$
e^x	2	$1{,}777 \cdot 10^{-2}$	$3{,}565 \cdot 10^{-2}$	$5{,}793 \cdot 10^{-4}$
	4	$4{,}467 \cdot 10^{-3}$	$8{,}940 \cdot 10^{-3}$	$3{,}701 \cdot 10^{-5}$
	8	$1{,}118 \cdot 10^{-3}$	$2{,}237 \cdot 10^{-3}$	$2{,}326 \cdot 10^{-6}$
	16	$2{,}796 \cdot 10^{-4}$	$5{,}593 \cdot 10^{-4}$	$1{,}456 \cdot 10^{-7}$
	32	$6{,}992 \cdot 10^{-5}$	$1{,}398 \cdot 10^{-4}$	$9{,}103 \cdot 10^{-9}$
	64	$1{,}748 \cdot 10^{-5}$	$3{,}496 \cdot 10^{-5}$	$5{,}690 \cdot 10^{-10}$
	128	$4{,}370 \cdot 10^{-6}$	$8{,}740 \cdot 10^{-6}$	$3{,}556 \cdot 10^{-11}$
	256	$1{,}092 \cdot 10^{-6}$	$2{,}185 \cdot 10^{-6}$	$2{,}223 \cdot 10^{-12}$
	512	$2{,}731 \cdot 10^{-7}$	$5{,}462 \cdot 10^{-7}$	$1{,}392 \cdot 10^{-13}$
	1024	$6{,}828 \cdot 10^{-8}$	$1{,}366 \cdot 10^{-7}$	$8{,}660 \cdot 10^{-15}$
$\sqrt{x}$	2	$1{,}635 \cdot 10^{-2}$	$6{,}311 \cdot 10^{-2}$	$2{,}860 \cdot 10^{-2}$
	4	$6{,}311 \cdot 10^{-3}$	$2{,}338 \cdot 10^{-2}$	$1{,}014 \cdot 10^{-2}$
	8	$2{,}366 \cdot 10^{-3}$	$8{,}536 \cdot 10^{-3}$	$3{,}587 \cdot 10^{-3}$
	16	$8{,}700 \cdot 10^{-4}$	$3{,}085 \cdot 10^{-3}$	$1{,}268 \cdot 10^{-3}$
	32	$3{,}160 \cdot 10^{-4}$	$1{,}108 \cdot 10^{-3}$	$4{,}485 \cdot 10^{-4}$

Продолжение таблицы 2

Функция	n	Прямоугольники	Трапеции	Симпсон
	64	$1{,}138 \cdot 10^{-4}$	$3{,}959 \cdot 10^{-4}$	$1{,}586 \cdot 10^{-4}$
	128	$4{,}077 \cdot 10^{-5}$	$1{,}410 \cdot 10^{-4}$	$5{,}606 \cdot 10^{-5}$
	256	$1{,}455 \cdot 10^{-5}$	$5{,}012 \cdot 10^{-5}$	$1{,}982 \cdot 10^{-5}$
	512	$5{,}176 \cdot 10^{-6}$	$1{,}779 \cdot 10^{-5}$	$7{,}008 \cdot 10^{-6}$
	1024	$1{,}838 \cdot 10^{-6}$	$6{,}304 \cdot 10^{-6}$	$2{,}478 \cdot 10^{-6}$

Зависимость погрешности от числа частей для функции $\sin x$ показана на рисунке 2, график построен библиотекой Lilaq [4]. В логарифмическом масштабе по обеим осям она изображается прямой, наклон которой равен порядку точности формулы со знаком минус.

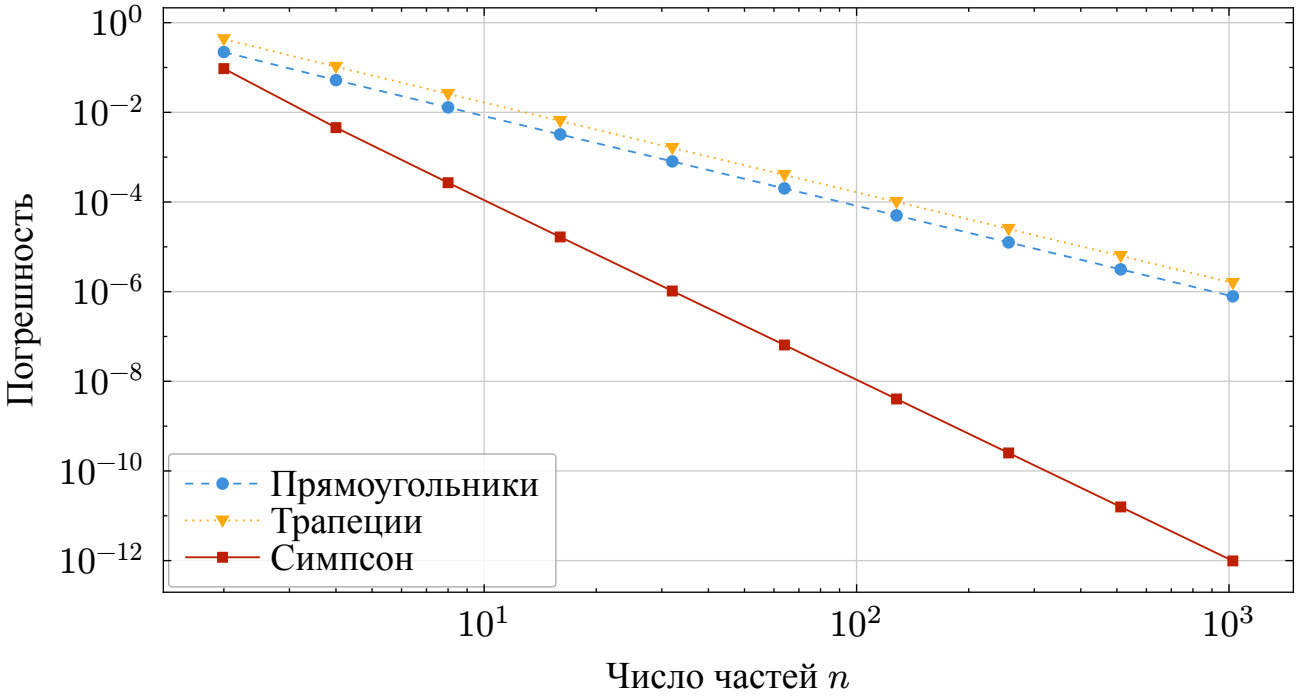


Рисунок 2 — Зависимость погрешности от числа частей для функции $\sin x$

Для функций $\sin x$ и e^x погрешность формул средних прямоугольников и трапеций при удвоении n уменьшается в 4 раза, а формулы Симпсона — в 16 раз, что совпадает с оценками из таблицы 1. Для функции $4/(1+x^2)$ формула Симпсона сходится быстрее, потому что третья производная этой функции на концах отрезка обращается в ноль и главный член её погрешности пропадает. Начиная с $n = 256$ погрешность становится меньше ошибки округления чисел двойной точности и в таблице записана нулём.

Для функции $\sqrt{x}$ погрешность всех трёх формул при удвоении n уменьшается лишь примерно в 2,8 раза, то есть как $h^{3/2}$. Производные этой функции не ограничены вблизи нуля, и оценки из таблицы 1 к ней неприменимы. Формула Симпсона здесь даже уступает формуле средних прямоугольников, потому что использует значение функции в самой точке $x = 0$.

Вывод

Для гладких функций экспериментальный порядок точности всех трёх формул совпал с теоретическим. При одинаковом числе вычислений функции формула Симпсона точнее остальных на несколько порядков, а формула средних прямоугольников вдвое точнее формулы трапеций. Если производные функции не ограничены, порядок точности всех формул падает, и преимущество формулы Симпсона пропадает.

ЗАКЛЮЧЕНИЕ

Цель работы, состоявшая в сравнении составных квадратурных формул средних прямоугольников, трапеций и Симпсона по точности, достигнута.

В ходе работы решены все поставленные задачи. Описаны три квадратурные формулы и оценки их погрешности. Формулы реализованы на языке Python и проверены на многочленах, которые они должны интегрировать точно. Погрешность вычислена для четырёх функций при числе частей от 2 до 1024 и сопоставлена с теоретическими оценками.

Формула Симпсона показала наибольшую точность на гладких функциях, и именно её стоит выбирать, когда подынтегральная функция имеет ограниченную четвёртую производную. Для функций с особенностью на конце отрезка, таких как $\sqrt{x}$, все три формулы сходятся медленно, и уменьшение шага помогает хуже, чем предсказывают оценки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы. 7-е изд. — Москва : БИНОМ. Лаборатория знаний, 2011. — 636 с.

2. Калиткин Н. Н. Численные методы. 2-е изд. — Санкт-Петербург : БХВ-Петербург, 2011. — 592 с.

3. Python 3 Documentation [Электронный ресурс]. — Режим доступа: https://docs.python.org/3/ (дата обращения: 10.10.2026).

4. Lilaq. Advanced data visualization in Typst [Электронный ресурс]. — Режим доступа: https://lilaq.org/ (дата обращения: 10.10.2026).

ПРИЛОЖЕНИЕ А

Полный текст программы приведён на листинге А.1. Программа вычисляет погрешность квадратурных формул и записывает её в CSV-файл, по которому построены таблица 2 и рисунок 2. При запуске с ключом `--test` программа вместо этого выполняет функциональные тесты.

```
"""Вычисление определённых интегралов методами средних прямоугольников, трапеций и Симпсона.

Программа считает абсолютную погрешность каждого метода для нескольких подынтегральных
    функций и разного числа отрезков разбиения и записывает результаты в CSV-файл.

Использование:
    python integrate.py [--output errors.csv] [--max-power 10]
    python integrate.py --test
"""

import argparse
import csv
import math
import sys
from typing import Callable, NamedTuple

Function = Callable[[float], float]


class Integrand(NamedTuple):
    """Подынтегральная функция с отрезком интегрирования и точным значением интеграла."""

    name: str
    f: Function
    a: float
    b: float
    exact: float


INTEGRANDS = [
    Integrand("4/(1+x^2)", lambda x: 4 / (1 + x * x), 0.0, 1.0, math.pi),
    Integrand("sin x", math.sin, 0.0, math.pi, 2.0),
    Integrand("e^x", math.exp, 0.0, 1.0, math.e - 1),
    Integrand("sqrt x", math.sqrt, 0.0, 1.0, 2 / 3),
]


def rectangles(f: Function, a: float, b: float, n: int) -> float:
    """Составная формула средних прямоугольников на n отрезках."""
    h = (b - a) / n
    return h * sum(f(a + (i + 0.5) * h) for i in range(n))


```

Продолжение листинга А.1

```
43 def trapezoids(f: Function, a: float, b: float, n: int) -> float:
44     """Составная формула трапеций на n отрезках."""
45     h = (b - a) / n
46     inner = sum(f(a + i * h) for i in range(1, n))
47     return h * ((f(a) + f(b)) / 2 + inner)
48
49
50 def simpson(f: Function, a: float, b: float, n: int) -> float:
51     """Составная формула Симпсона на n отрезках, n должно быть чётным."""
52     if n % 2 != 0:
53         raise ValueError(f"для формулы Симпсона нужно чётное число отрезков, получено {n}")
54     h = (b - a) / n
55     odd = sum(f(a + i * h) for i in range(1, n, 2))
56     even = sum(f(a + i * h) for i in range(2, n, 2))
57     return h / 3 * (f(a) + f(b) + 4 * odd + 2 * even)
58
59
60 METHODS = [rectangles, trapezoids, simpson]
61
62
63 def errors(integrand: Integrand, n: int) -> list[float]:
64     """Абсолютные погрешности всех методов для функции integrand на n отрезках."""
65     f, a, b = integrand.f, integrand.a, integrand.b
66     return [abs(method(f, a, b, n) - integrand.exact) for method in METHODS]
67
68
69 def write_errors(path: str, max_power: int) -> None:
70     """Записывает погрешности для n = 2, 4, ..., 2^max_power в CSV-файл path."""
71     with open(path, "w", newline="") as file:
72         writer = csv.writer(file)
73         for index, integrand in enumerate(INTEGRANDS):
74             for power in range(1, max_power + 1):
75                 n = 2**power
76                 writer.writerow([index, n, *(f"{e:.3e}" for e in errors(integrand, n))])
77
78
79 def run_tests() -> bool:
80     """Проверяет методы на многочленах, которые они интегрируют точно, и возвращает признак
       успеха."""
81     cases = [
82         ("прямоугольники, линейная функция", rectangles, lambda x: 3 * x + 1, 2, 8.0),
83         ("трапеции, линейная функция", trapezoids, lambda x: 3 * x + 1, 2, 8.0),
84         ("Симпсон, кубическая функция", simpson, lambda x: x**3 - x, 2, 2.0),
85         ("Симпсон, квадратичная функция", simpson, lambda x: x * x, 4, 8 / 3),
86     ]
87     ok = True
88     for name, method, f, n, expected in cases:
89         result = method(f, 0.0, 2.0, n)
90         passed = math.isclose(result, expected, rel_tol=1e-12)
91         ok = ok and passed
92         print(f"{'OK  ' if passed else 'FAIL'} {name}: {result} (ожидалось {expected})")
```

Продолжение листинга А.1

```
    try:
        simpson(math.sin, 0.0, 1.0, 3)
        print("FAIL Симпсон, нечётное число отрезков: нет ошибки")
        ok = False
    except ValueError:
        print("OK   Симпсон, нечётное число отрезков: ошибка")
    return ok


def main() -> int:
    """Разбирает аргументы командной строки, выполняет тесты или замеры и возвращает код
        завершения."""
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--output", default="errors.csv", help="файл для записи
        погрешностей")
    parser.add_argument("--max-power", type=int, default=10, help="наибольшая степень двойки
        в числе отрезков")
    parser.add_argument("--test", action="store_true", help="запустить проверки вместо
        замеров")
    args = parser.parse_args()

    if args.test:
        return 0 if run_tests() else 1
    write_errors(args.output, args.max_power)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Листинг А.1 — Программа вычисления погрешности квадратурных формул